http://localhost:3000/dashboard

100	100	100	100
Performance	Accessibility	Best Practices	SEO

100

Performance



Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.

▲ 0–49 50–89 90–100

METRICS Expand view

First Contentful Paint	Largest Contentful Paint
0.3 s	0.5 s
Total Blocking Time	**Cumulative Layout Shift**
0 ms	0.001
Speed Index	
0.4 s	



Show audits relevant to: All FCP LCP CLS

INSIGHTS

▲ Render-blocking requests — Est savings of 150 ms

Requests are blocking the page's initial render, which may delay LCP. Deferring or inlining can move these network requests out of the critical path. FCP LCP Unscored

URL	Transfer Size	Duration
localhost 1st Party	**8.1 KiB**	**60 ms**
...chunks/0h9iqwqlqk7g7.css (localhost)	8.1 KiB	60 ms

▲ Network dependency tree

Avoid chaining critical requests by reducing the length of chains, reducing the download size of resources, or deferring the download of unnecessary resources to improve page load. LCP Unscored

Maximum critical path latency: **177 ms**

Initial Navigation

/dashboard (localhost) - **98 ms,** 7.35 KiB

...chunks/0h9iqwqlqk7g7.css (localhost) - **116 ms,** 8.06 KiB

...media/inter-latin-400-normal.2qdljeg3s-lsl.woff2 (localhost) - **177 ms,** 23.59 KiB

Preconnected origins

preconnect hints help the browser establish a connection earlier in the page load, saving time when the first request for that origin is made. The following are the origins that the page preconnected to.

no origins were preconnected

Preconnect candidates

Add preconnect hints to your most important origins, but try to use no more than 4.

No additional origins are good candidates for preconnecting

Legacy JavaScript — Est savings of 14 KiB

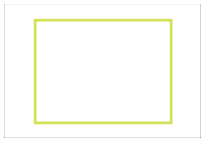
Polyfills and transforms enable older browsers to use new JavaScript features. However, many aren't necessary for modern browsers. Consider modifying your JavaScript build process to not transpile Baseline features, unless you know you must support older browsers. Learn why most sites can deploy ES6+ code without transpiling FCP LCP Unscored

URL	Wasted bytes
localhost 1st Party	13.5 KiB
...chunks/3wbm-994r45kr.js (localhost)	13.5 KiB

3wbm-994r45kr.js:1	`Array.prototype.at`
3wbm-994r45kr.js:1	`Array.prototype.flat`
3wbm-994r45kr.js:1	`Array.prototype.flatMap`
3wbm-994r45kr.js:1	`Object.fromEntries`
3wbm-994r45kr.js:1	`Object.hasOwn`
3wbm-994r45kr.js:1	`String.prototype.trimEnd`
3wbm-994r45kr.js:1	`String.prototype.trimStart`

Layout shift culprits

Layout shifts occur when elements move absent any user interaction. Investigate the causes of layout shifts, such as elements being added, removed, or their fonts changing as the page loads. CLS Unscored

Element	Layout shift score
Total	0.001
 div.flex.flex-col	0.001
div.flex.flex-col	0.000

LCP breakdown

Each subpart has specific improvement strategies. Ideally, most of the LCP time should be spent on loading the resources, not within delays. LCP Unscored

Subpart	Duration
Time to first byte	20 ms
Element render delay	170 ms

 p.mt-2.text-display.text-inverse

These insights are also available in the Chrome DevTools Performance Panel - record a trace to view more detailed information.

DIAGNOSTICS

▲ Page prevented back/forward cache restoration — 3 failure reasons

Many navigations are performed by going back to a previous page, or forwards again. The back/forward cache (bfcache) can speed up these return navigations. Learn more about the bfcache Unscored

Failure reason	Failure type
Pages with cache-control:no-store header cannot enter back/forward cache. /dashboard (localhost)	Actionable
Pages whose main resource has cache-control:no-store cannot enter back/forward cache. /dashboard (localhost)	Not actionable
Back/forward cache is disabled because some JavaScript network request received resource with `Cache-Control: no-store` header. /dashboard (localhost)	Not actionable

Reduce unused JavaScript — Est savings of 65 KiB

Reduce unused JavaScript and defer loading scripts until they are required to decrease bytes consumed by network activity. Learn how to reduce unused JavaScript. FCP LCP Unscored

URL	Transfer Size	Est Savings
localhost 1st Party	**162.7 KiB**	**64.7 KiB**
…chunks/2ce14t0rx4b4t.js (localhost)	92.4 KiB	42.5 KiB
…chunks/3wbm-994r45kr.js (localhost)	70.4 KiB	22.2 KiB

More information about the performance of your application. These numbers don't directly affect the Performance score.

PASSED AUDITS (22) Hide

Use efficient cache lifetimes

A long cache lifetime can speed up repeat visits to your page. Learn more about caching. FCP LCP Unscored

Document request latency

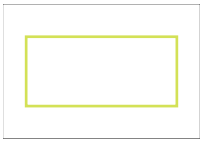
Your first network request is the most important. Reduce its latency by avoiding redirects, ensuring a fast server response, and enabling text compression. FCP LCP Unscored

- Avoids redirects
- Server responds quickly (observed 21 ms)
- Applies text compression

Optimize DOM size

A large DOM can increase the duration of style calculations and layout reflows, impacting page responsiveness. A large DOM will also increase memory usage. Learn how to avoid an excessive DOM size. Unscored

Statistic	Element		Value
Total elements			370
DOM depth		path.recharts-rectangle	21

Statistic	Element	Value
Most children	 svg.recharts-surface	15

Duplicated JavaScript

Remove large, duplicate JavaScript modules from bundles to reduce unnecessary bytes consumed by network activity. FCP LCP Unscored

Font display

Consider setting `font-display` to `swap` or `optional` to ensure text is consistently visible. `swap` can be further optimized to mitigate layout shifts with font metric overrides. Unscored

Forced reflow

A forced reflow occurs when JavaScript queries geometric properties (such as `offsetWidth`) after styles have been invalidated by a change to the DOM state. This can result in poor performance. Learn more about forced reflows and possible mitigations. Unscored

Improve image delivery

Reducing the download time of images can improve the perceived load time of the page and LCP. Learn more about optimizing image size FCP LCP Unscored

INP breakdown

Start investigating how to improve INP by looking at the longest subpart. Unscored

LCP request discovery

Optimize LCP by making the LCP image discoverable from the HTML immediately, and avoiding lazy-loading Unscored

Modern HTTP

HTTP/2 and HTTP/3 offer many benefits over HTTP/1.1, such as multiplexing. Learn more about using modern HTTP. FCP LCP Unscored

3rd parties

3rd party code can significantly impact load performance. Reduce and defer loading of 3rd party code to prioritize your page's content. Unscored

Optimize viewport for mobile

Tap interactions may be delayed by up to 300 ms if the viewport is not optimized for mobile. Unscored

meta

Minify CSS

Minifying CSS files can reduce network payload sizes. Learn how to minify CSS. FCP LCP Unscored

Minify JavaScript

Minifying JavaScript files can reduce payload sizes and script parse time. Learn how to minify JavaScript. FCP LCP Unscored

Reduce unused CSS

Reduce unused rules from stylesheets and defer CSS not used for above-the-fold content to decrease bytes consumed by network activity. Learn how to reduce unused CSS. FCP LCP Unscored

Avoids enormous network payloads — Total size was 347 KiB

Large network payloads cost users real money and are highly correlated with long load times. Learn how to reduce payload sizes. Unscored

URL	Transfer Size
localhost 1st Party	**297.5 KiB**
...chunks/2ce14t0rx4b4t.js (localhost)	92.9 KiB
...chunks/3wbm-994r45kr.js (localhost)	70.9 KiB
...chunks/11o6mj_wnqju8.js (localhost)	44.9 KiB
...media/inter-latin-400-normal.2qdljeg3s-lsl.woff2 (localhost)	23.6 KiB
...chunks/3ec_xmde2-jw6.js (localhost)	14.3 KiB
...chunks/25zxpn_ykovce.js (localhost)	13.8 KiB
...chunks/36telqiutlmc6.js (localhost)	10.0 KiB
...chunks/14gnlgb1qg7gz.js (localhost)	9.7 KiB
...chunks/00rnq87v0zc5b.js (localhost)	9.2 KiB
...chunks/3h2m0v0uuck8b.js (localhost)	8.3 KiB

User Timing marks and measures

Consider instrumenting your app with the User Timing API to measure your app's real-world performance during key user experiences. Learn more about User Timing marks. Unscored

JavaScript execution time — 0.3 s

Consider reducing the time spent parsing, compiling, and executing JS. You may find delivering smaller JS payloads helps with this. Learn how to reduce Javascript execution time. TBT Unscored

URL	Total CPU Time	Script Evaluation	Script Parse
localhost 1st Party	**539 ms**	**288 ms**	**33 ms**
...chunks/3wbm-994r45kr.js (localhost)	268 ms	248 ms	4 ms
/dashboard (localhost)	195 ms	6 ms	21 ms
...chunks/2ce14t0rx4b4t.js (localhost)	76 ms	34 ms	8 ms
Unattributable	**109 ms**	**12 ms**	**0 ms**
`Unattributable`	109 ms	12 ms	0 ms

Minimizes main-thread work — 0.7 s

Consider reducing the time spent parsing, compiling and executing JS. You may find delivering smaller JS payloads helps with this. Learn how to minimize main-thread work TBT Unscored

Category	Time Spent
Script Evaluation	326 ms
Other	223 ms

Category	Time Spent
Style & Layout	59 ms
Script Parsing & Compilation	40 ms
Rendering	30 ms
Garbage Collection	5 ms
Parse HTML & CSS	2 ms

Avoid long main-thread tasks

Lists the longest tasks on the main thread, useful for identifying worst contributors to input delay. Learn how to avoid long main-thread tasks TBT Unscored

Avoid non-composited animations

Animations which are not composited can be janky and increase CLS. Learn how to avoid non-composited animations CLS Unscored

Image elements have explicit `width` and `height`

Set an explicit width and height on image elements to reduce layout shifts and improve CLS. Learn how to set image dimensions CLS Unscored

100

Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.

NAMES AND LABELS

No form fields have multiple labels

Form fields with multiple labels can be confusingly announced by assistive technologies like screen readers which use either the first, the last, or all of the labels. Learn how to use form labels. Unscored

Failing Elements

input#attachment-input.sr-only

label.text-body-semibold.text-primary-900



label.flex.items-center.gap-3.h-12.px-4.rounded-md.border.border-primary-900.bg-inverse.cursor-poi...

These are opportunities to improve the semantics of the controls in your application. This may enhance the experience for users of assistive technology, like a screen reader.

ADDITIONAL ITEMS TO MANUALLY CHECK (10) Hide

Interactive controls are keyboard focusable

Custom interactive controls are keyboard focusable and display a focus indicator. Learn how to make custom controls focusable. Unscored

- Interactive elements indicate their purpose and state

Interactive elements, such as links and buttons, should indicate their state and be distinguishable from non-interactive elements. Learn how to decorate interactive elements with affordance hints. Unscored

- The page has a logical tab order

Tabbing through the page follows the visual layout. Users cannot focus elements that are offscreen. Learn more about logical tab ordering. Unscored

- Visual order on the page follows DOM order

DOM order matches the visual order, improving navigation for assistive technology. Learn more about DOM and visual ordering. Unscored

- User focus is not accidentally trapped in a region

A user can tab into and out of any control or region without accidentally trapping their focus. Learn how to avoid focus traps. Unscored

- The user's focus is directed to new content added to the page

If new content, such as a dialog, is added to the page, the user's focus is directed to it. Learn how to direct focus to new content. Unscored

- HTML5 landmark elements are used to improve navigation

Landmark elements (`<main>`, `<nav>`, etc.) are used to improve the keyboard navigation of the page for assistive technology. Learn more about landmark elements. Unscored

- Offscreen content is hidden from assistive technology

Offscreen content is hidden with display: none or aria-hidden=true. Learn how to properly hide offscreen content. Unscored

- Custom controls have associated labels

Custom interactive controls have associated labels, provided by aria-label or aria-labelledby. Learn more about custom controls and labels. Unscored

- Custom controls have ARIA roles

Custom interactive controls have appropriate ARIA roles. Learn how to add roles to custom controls. Unscored

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.

PASSED AUDITS (24) Hide

- `[aria-*]` attributes match their roles

Each ARIA `role` supports a specific subset of `aria-*` attributes. Mismatching these invalidates the `aria-*` attributes. Learn how to match ARIA attributes to their roles.

- `[aria-hidden="true"]` is not present on the document `<body>`

Assistive technologies, like screen readers, work inconsistently when `aria-hidden="true"` is set on the document `<body>`. Learn how `aria-hidden` affects the document body.

- `[role]`s have all required `[aria-*]` attributes

Some ARIA roles have required attributes that describe the state of the element to screen readers. Learn more about roles and required attributes.

- `[role]` values are valid

ARIA roles must have valid values in order to perform their intended accessibility functions. Learn more about valid ARIA roles.

`[aria-*]` attributes have valid values

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid values. Learn more about valid values for ARIA attributes.

`[aria-*]` attributes are valid and not misspelled

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid names. Learn more about valid ARIA attributes.

Buttons have an accessible name

When a button doesn't have an accessible name, screen readers announce it as "button", making it unusable for users who rely on screen readers. Learn how to make buttons more accessible.

Form elements have associated labels

Labels ensure that form controls are announced properly by assistive technologies, like screen readers. Learn more about form element labels.

`[user-scalable="no"]` is not used in the `<meta name="viewport">` element and the `[maximum-scale]` attribute is not less than 5.

Disabling zooming is problematic for users with low vision who rely on screen magnification to properly see the contents of a web page. Learn more about the viewport meta tag.

ARIA attributes are used as specified for the element's role

Some ARIA attributes are only allowed on an element under certain conditions. Learn more about conditional ARIA attributes.

`[aria-hidden="true"]` elements do not contain focusable descendents

Focusable descendents within an `[aria-hidden="true"]` element prevent those interactive elements from being available to users of assistive technologies like screen readers. Learn how `aria-hidden` affects focusable elements.

Elements use only permitted ARIA attributes

Using ARIA attributes in roles where they are prohibited can mean that important information is not communicated to users of assistive technologies. Learn more about prohibited ARIA roles.

Background and foreground colors have a sufficient contrast ratio

Low-contrast text is difficult or impossible for many users to read. Learn how to provide sufficient color contrast.

Document has a `<title>` element

The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

`<html>` element has a `[lang]` attribute

If a page doesn't specify a `lang` attribute, a screen reader assumes that the page is in the default language that the user chose when setting up the screen reader. If the page isn't actually in the default language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

`<html>` element has a valid value for its `[lang]` attribute

Specifying a valid BCP 47 language helps screen readers announce text properly. Learn how to use the `lang` attribute.

Links have a discernible name

Link text (and alternate text for images, when used as links) that is discernible, unique, and focusable improves the navigation experience for screen reader users. Learn how to make links accessible.

Lists contain only `<li>` elements and script supporting elements (`<script>` and `<template>`).

Screen readers have a specific way of announcing lists. Ensuring proper list structure aids screen reader output. Learn more about proper list structure.

List items (`<li>`) are contained within `<ul>`, `<ol>` or `<menu>` parent elements

Screen readers require list items (`<li>`) to be contained within a parent `<ul>`, `<ol>` or `<menu>` to be announced properly. Learn more about proper list structure.

Touch targets have sufficient size and spacing.

Touch targets with sufficient size and spacing help users who may have difficulty targeting small controls to activate the targets. Learn more about touch targets.

Cells in a `<table>` element that use the `[headers]` attribute refer to table cells within the same table.

Screen readers have features to make navigating tables easier. Ensuring `<td>` cells using the `[headers]` attribute only refer to other cells in the same table may improve the experience for screen reader users. Learn more about the `headers` attribute.

Heading elements appear in a sequentially-descending order

Properly ordered headings that do not skip levels convey the semantic structure of the page, making it easier to navigate and understand when using assistive technologies. Learn more about heading order.

Document has a main landmark.

One main landmark helps screen reader users navigate a web page. Learn more about landmarks.

Deprecated ARIA roles were not used

Deprecated ARIA roles may not be processed correctly by assistive technology. Learn more about deprecated ARIA roles.

NOT APPLICABLE (38) Hide

`[accesskey]` values are unique

Access keys let users quickly focus a part of the page. For proper navigation, each access key must be unique. Learn more about access keys. Unscored

`button`, `link`, and `menuitem` elements have accessible names

When an element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to make command elements more accessible. Unscored

Elements with `role="dialog"` or `role="alertdialog"` have accessible names.

ARIA dialog elements without accessible names may prevent screen readers users from discerning the purpose of these elements. Learn how to make ARIA dialog elements more accessible. Unscored

ARIA input fields have accessible names

When an input field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about input field labels. Unscored

ARIA `meter` elements have accessible names

When a meter element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `meter` elements. Unscored

ARIA `progressbar` elements have accessible names

When a `progressbar` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to label `progressbar` elements. Unscored

- Elements with an ARIA `[role]` that require children to contain a specific `[role]` have all required children.

  Some ARIA parent roles must contain specific child roles to perform their intended accessibility functions. Learn more about roles and required children elements. Unscored

- `[role]`s are contained by their required parent element

  Some ARIA child roles must be contained by specific parent roles to properly perform their intended accessibility functions. Learn more about ARIA roles and required parent element. Unscored

- Elements with the `role=text` attribute do not have focusable descendents.

  Adding `role=text` around a text node split by markup enables VoiceOver to treat it as one phrase, but the element's focusable descendents will not be announced. Learn more about the `role=text` attribute. Unscored

- ARIA toggle fields have accessible names

  When a toggle field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about toggle fields. Unscored

- ARIA `tooltip` elements have accessible names

  When a tooltip element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `tooltip` elements. Unscored

- ARIA `treeitem` elements have accessible names

  When a `treeitem` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about labeling `treeitem` elements. Unscored

- The page contains a heading, skip link, or landmark region

  Adding ways to bypass repetitive content lets keyboard users navigate the page more efficiently. Learn more about bypass blocks. Unscored

- `<dl>`'s contain only properly-ordered `<dt>` and `<dd>` groups, `<script>`, `<template>` or `<div>` elements.

  When definition lists are not properly marked up, screen readers may produce confusing or inaccurate output. Learn how to structure definition lists correctly. Unscored

- Definition list items are wrapped in `<dl>` elements

  Definition list items (`<dt>` and `<dd>`) must be wrapped in a parent `<dl>` element to ensure that screen readers can properly announce them. Learn how to structure definition lists correctly. Unscored

- ARIA IDs are unique

  The value of an ARIA ID must be unique to prevent other instances from being overlooked by assistive technologies. Learn how to fix duplicate ARIA IDs. Unscored

- `<frame>` or `<iframe>` elements have a title

  Screen reader users rely on frame titles to describe the contents of frames. Learn more about frame titles. Unscored

- `<html>` element has an `[xml:lang]` attribute with the same base language as the `[lang]` attribute.

  If the webpage does not specify a consistent language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute. Unscored

- Image elements have `[alt]` attributes

  Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute. Unscored

- Input buttons have discernible text.

Adding discernable and accessible text to input buttons may help screen reader users understand the purpose of the input button. Learn more about input buttons. Unscored

- `<input type="image">` elements have `[alt]` text

When an image is being used as an `<input>` button, providing alternative text can help screen reader users understand the purpose of the button. Learn about input image alt text. Unscored

- Links are distinguishable without relying on color.

Low-contrast text is difficult or impossible for many users to read. Link text that is discernible improves the experience for users with low vision. Learn how to make links distinguishable. Unscored

- The document does not use `<meta http-equiv="refresh">`

Users do not expect a page to refresh automatically, and doing so will move focus back to the top of the page. This may create a frustrating or confusing experience. Learn more about the refresh meta tag. Unscored

- `<object>` elements have alternate text

Screen readers cannot translate non-text content. Adding alternate text to `<object>` elements helps screen readers convey meaning to users. Learn more about alt text for `object` elements. Unscored

- Select elements have associated label elements.

Form elements without effective labels can create frustrating experiences for screen reader users. Learn more about the `select` element. Unscored

- Skip links are focusable.

Including a skip link can help users skip to the main content to save time. Learn more about skip links. Unscored

- No element has a `[tabindex]` value greater than 0

A value greater than 0 implies an explicit navigation ordering. Although technically valid, this often creates frustrating experiences for users who rely on assistive technologies. Learn more about the `tabindex` attribute. Unscored

- `<th>` elements and elements with `[role="columnheader"/"rowheader"]` have data cells they describe.

Screen readers have features to make navigating tables easier. Ensuring table headers always refer to some set of cells may improve the experience for screen reader users. Learn more about table headers. Unscored

- `[lang]` attributes have a valid value

Specifying a valid BCP 47 language on elements helps ensure that text is pronounced correctly by a screen reader. Learn how to use the `lang` attribute. Unscored

- `<video>` elements contain a `<track>` element with `[kind="captions"]`

When a video provides a caption it is easier for deaf and hearing impaired users to access its information. Learn more about video captions. Unscored

- `autocomplete` attributes are used correctly

The `autocomplete` attribute values must be valid and correctly applied for screen readers to function correctly. Learn more about valid autocomplete values. Unscored

- Elements with `role="none"` or `role="presentation"` do not have conflicts

There are certain cases where the semantic role of an element with `role="none"` or `role="presentation"` does not resolve to none or presentation. To ensure the element remains removed from the accessibility tree, you should not add any global ARIA attributes to the element or make it focusable. Learn more about presentation role conflict. Unscored

- SVG elements with an `img` role have an accessible text alternative

Ensures SVG elements with an `img`, `graphics-document` or `graphics-symbol` role have an accessible text alternative. Learn more about SVG alt text. Unscored

- Tables have different content in the summary attribute and `<caption>`.

The summary attribute should describe the table structure, while `<caption>` should have the onscreen title. Accurate table mark-up helps users of screen readers. Learn more about summary and caption. Unscored

- All heading elements contain content.

A heading with no content or inaccessible text prevent screen reader users from accessing information on the page's structure. Learn more about headings. Unscored

- Uses ARIA roles only on compatible elements

Many HTML elements can only be assigned certain ARIA roles. Using ARIA roles where they are not allowed can interfere with the accessibility of the web page. Learn more about ARIA roles. Unscored

- Image elements do not have `[alt]` attributes that are redundant text.

Informative elements should aim for short, descriptive alternative text. Alternative text that is exactly the same as the text adjacent to the link or image is potentially confusing for screen reader users, because the text will be read twice. Learn more about the `alt` attribute. Unscored

- Identical links have the same purpose.

Links with the same destination should have the same description, to help users understand the link's purpose and decide whether to follow it. Learn more about identical links. Unscored

100

Best Practices

TRUST AND SAFETY

- Ensure CSP is effective against XSS attacks

A strong Content Security Policy (CSP) significantly reduces the risk of cross-site scripting (XSS) attacks. Learn how to use a CSP to prevent XSS Unscored

Description	Directive	Severity
Consider adding `'unsafe-inline'` (ignored by browsers supporting nonces/hashes) to be backward compatible with older browsers.	`script-src`	Medium
Consider adding https: and http: URL schemes (ignored by browsers supporting `'strict-dynamic'`) to be backward compatible with older browsers.	`script-src`	Medium

- Use a strong HSTS policy

Deployment of the HSTS header significantly reduces the risk of downgrading HTTP connections and eavesdropping attacks. A rollout in stages, starting with a low max-age is recommended. Learn more about using a strong HSTS policy. Unscored

Description	Directive	Severity
No HSTS header found		High

- Ensure proper origin isolation with COOP

The Cross-Origin-Opener-Policy (COOP) can be used to isolate the top-level window from other documents such as pop-ups. Learn more about deploying the COOP header. Unscored

Description	Directive	Severity
No COOP header found		High

Mitigate DOM-based XSS with Trusted Types

The `require-trusted-types-for` directive in the `Content-Security-Policy` (CSP) header instructs user agents to control the data passed to DOM XSS sink functions. Learn more about mitigating DOM-based XSS with Trusted Types. Unscored

Description	Severity
No `Content-Security-Policy` header with Trusted Types directive found	High

BROWSER COMPATIBILITY

Baseline Features

Lists web features used on the page and their Baseline status as of 2026-09-17. Learn more about Baseline. Unscored

Web-features	Baseline Status	Source
localhost 1st Party		
page-visibility-state	Limited Availability	dashboard:2
interaction-contentful-paint	Limited Availability	dashboard:2
soft-navigations	Limited Availability	dashboard:2
long-animation-frames	Limited Availability	dashboard:2
layout-instability	Limited Availability	dashboard:2
requestidlecallback	Limited Availability	dashboard:2
referencetarget	Limited Availability	3wbm-994r45kr.js:1
largest-contentful-paint	Newly Available (2025-12-12)	dashboard:2
event-timing	Newly Available (2025-12-12)	dashboard:2
scrollend	Newly Available (2025-12-12)	3wbm-994r45kr.js:1
scrollbar-gutter	Newly Available (2024-12-11)	dashboard:-1
fetch-priority	Newly Available (2024-10-29)	dashboard:-1
relative-color	Newly Available (2024-09-16)	dashboard:-1
starting-style	Newly Available (2024-08-06)	dashboard:-1
registered-custom-properties	Newly Available (2024-07-09)	dashboard:-1
has	Widely Available (2023-12-19)	dashboard:-1
aria-attribute-reflection	Widely Available (2023-10-24)	3wbm-994r45kr.js:1

Web-features	Baseline Status	Source
contain-intrinsic-size	Widely Available (2023-09-18)	dashboard:-1
overflow	Widely Available (2023-09-18)	dashboard/_next/stat…/2ce14t0rx4b4t.js:1
color-mix	Widely Available (2023-05-09)	dashboard:-1
lab	Widely Available (2023-05-09)	dashboard:-1
inert	Widely Available (2023-04-11)	dashboard:-1
outline	Widely Available (2023-03-27)	dashboard:-1
forced-colors	Widely Available (2022-09-12)	dashboard:-1
overflow-clip	Widely Available (2022-09-12)	dashboard:-1
individual-transforms	Widely Available (2022-08-05)	dashboard:-1
cascade-layers	Widely Available (2022-03-14)	dashboard:-1
focus-visible	Widely Available (2022-03-14)	dashboard:-1
appearance	Widely Available (2022-03-14)	dashboard:-1
broadcast-channel	Widely Available (2022-03-14)	36telqiutlmc6.js:1
logical-properties	Widely Available (2021-09-20)	dashboard:-1
file-selector-button	Widely Available (2021-04-26)	dashboard:-1
weak-references	Widely Available (2021-04-26)	dashboard:2
where	Widely Available (2021-01-21)	dashboard:-1
is	Widely Available (2021-01-21)	dashboard:-1
not	Widely Available (2021-01-21)	dashboard:-1
clip-path	Widely Available (2021-01-21)	dashboard:-1
nullish-coalescing	Widely Available (2020-09-16)	dashboard:1
logical-assignments	Widely Available (2020-09-16)	dashboard:1
aborting	Widely Available (2019-03-25)	dashboard:2

Web-features	Baseline Status	Source
intersection-observer	Widely Available (2019-03-25)	11o6mj_wnqju8.js:1
flexbox	Widely Available (2015-09-30)	dashboard:-1
text-indent	Widely Available (2015-07-29)	dashboard:-1
request-animation-frame	Widely Available (2015-07-29)	3ec_xmde2-jw6.js:1
Unattributable		
referrer-policy	Widely Available (2021-09-20)	

PASSED AUDITS (13) Hide

Uses HTTPS

All sites should be protected with HTTPS, even ones that don't handle sensitive data. This includes avoiding mixed content, where some resources are loaded over HTTP despite the initial request being served over HTTPS. HTTPS prevents intruders from tampering with or passively listening in on the communications between your app and your users, and is a prerequisite for HTTP/2 and many new web platform APIs. Learn more about HTTPS.

Avoids deprecated APIs

Deprecated APIs will eventually be removed from the browser. Learn more about deprecated APIs.

Avoids third-party cookies

Third-party cookies may be blocked in some contexts. Learn more about preparing for third-party cookie restrictions.

Allows users to paste into input fields

Preventing input pasting is a bad practice for the UX, and weakens security by blocking password managers.Learn more about user-friendly input fields.

Avoids requesting the geolocation permission on page load

Users are mistrustful of or confused by sites that request their location without context. Consider tying the request to a user action instead. Learn more about the geolocation permission. Also consider using the `<geolocation>` element.

Avoids requesting the notification permission on page load

Users are mistrustful of or confused by sites that request to send notifications without context. Consider tying the request to user gestures instead. Learn more about responsibly getting permission for notifications.

Displays images with correct aspect ratio

Image display dimensions should match natural aspect ratio. Learn more about image aspect ratio.

Serves images with appropriate resolution

Image natural dimensions should be proportional to the display size and the pixel ratio to maximize image clarity. Learn how to provide responsive images.

Page has the HTML doctype

Specifying a doctype prevents the browser from switching to quirks-mode. Learn more about the doctype declaration.

Properly defines charset

A character encoding declaration is required. It can be done with a `<meta>` tag in the first 1024 bytes of the HTML or in the Content-Type HTTP response header. Learn more about declaring the character encoding.

No browser errors logged to the console

Errors logged to the console indicate unresolved problems. They can come from network request failures and other browser concerns. Learn more about this errors in console diagnostic audit

No issues in the `Issues` panel in Chrome Devtools

Issues logged to the `Issues` panel in Chrome Devtools indicate unresolved problems. They can come from network request failures, insufficient security controls, and other browser concerns. Open up the Issues panel in Chrome DevTools for more details on each issue.

Page has valid source maps

Source maps translate minified code to the original source code. This helps developers debug in production. In addition, Lighthouse is able to provide further insights. Consider deploying source maps to take advantage of these benefits. Learn more about source maps. Unscored

NOT APPLICABLE (3) Hide

Redirects HTTP traffic to HTTPS

Make sure that you redirect all HTTP traffic to HTTPS in order to enable secure web features for all your users. Learn more. Unscored

Mitigate clickjacking with XFO or CSP

The `X-Frame-Options` (XFO) header or the `frame-ancestors` directive in the `Content-Security-Policy` (CSP) header control where a page can be embedded. These can mitigate clickjacking attacks by blocking some or all sites from embedding the page. Learn more about mitigating clickjacking. Unscored

Detected JavaScript libraries

All front-end JavaScript libraries detected on the page. Learn more about this JavaScript library detection diagnostic audit. Unscored

100

SEO

These checks ensure that your page is following basic search engine optimization advice. There are many additional factors Lighthouse does not score here that may affect your search ranking, including performance on Core Web Vitals. Learn more about Google Search Essentials.

ADDITIONAL ITEMS TO MANUALLY CHECK (1) Hide

Structured data is valid

Run the Structured Data Testing Tool to validate structured data. Learn more about Structured Data. Unscored

Run these additional validators on your site to check additional SEO best practices.

PASSED AUDITS (7) Hide

Page isn't blocked from indexing

Search engines are unable to include your pages in search results if they don't have permission to crawl them. Learn more about crawler directives.

Document has a `<title>` element

The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

Document has a meta description

Meta descriptions may be included in search results to concisely summarize page content. Learn more about the meta description.

Page has successful HTTP status code

Pages with unsuccessful HTTP status codes may not be indexed properly. Learn more about HTTP status codes.

Links have descriptive text

Descriptive link text helps search engines understand your content. Learn how to make links more accessible.

Links are crawlable

Search engines may use `href` attributes on links to crawl websites. Ensure that the `href` attribute of anchor elements links to an appropriate destination, so more pages of the site can be discovered. Learn how to make links crawlable

Document has a valid `hreflang`

hreflang links tell search engines what version of a page they should list in search results for a given language or region. Learn more about `hreflang`.

NOT APPLICABLE (3)

Hide

robots.txt is valid

If your robots.txt file is malformed, crawlers may not be able to understand how you want your website to be crawled or indexed. Learn more about robots.txt. Unscored

Image elements have `[alt]` attributes

Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute. Unscored

Document has a valid `rel=canonical`

Canonical links suggest which URL to show in search results. Learn more about canonical links. Unscored

Captured at Nov 2, 2026, 11:27 AM GMT-3
Initial page load
Emulated Desktop with Lighthouse 13.5.0
Custom throttling
Single page session
Using Chromium 156.0.0.0 with devtools

Generated by **Lighthouse** 13.5.0 | File an issue